Satya-Score: How the System Works

Oct 13, 2026 · @Vivek

Overview

Satya-Score checks English and Hindi text one claim at a time and labels each claim **Verified**, **Uncertain** or **Hallucinated**, with the evidence behind it. It splits an answer into claims, sends each claim to the most trustworthy source that can settle it, and marks the wrong words when a claim is false.

The one-paragraph explanation: *A chatbot answer is broken into separate factual claims. Each claim is checked first against official statistics (World Bank, WHO) and the Wikidata knowledge graph, then against published fact-checks. Claims none of these settle are checked against Wikipedia passages by an inference model and, in Version 2, by an open-source LLM. The LLM's opinion counts only when the evidence agrees with it.*

	Version 1 (default)	Version 2 (LLM cross-check)
Who decides claims no structured source covers	Inference model on Wikipedia passages	Inference model **and** the LLM gpt-oss-20b, which must agree
False items caught (620-item test)	25%	47%
False items wrongly passed as Verified	22%	6%
False alarms on true items	11%	13%
Time for a 5–6 claim answer	about 5 s	about 16–46 s
How it is switched on	default	`start_v2.ps1` or `Start Satya-Score.bat`

Version 2 is what the demo, the IEEE paper v2, the synopsis v2 and the project report v2 describe. Version 1 is still the code default, because Version 2 met 3 of the 4 acceptance criteria set in advance (it reached 1.85× the catches, not 2×).

Objectives (the same wording as in the synopsis and the project report):

1. To analyse hallucination in LLM answers.
2. To compare existing hallucination-detection methods.
3. To design a claim-level detection framework for English and Hindi.

4. To integrate an LLM with retrieved evidence so that neither decides alone.
5. To develop a secure web application that explains each verdict.
6. To evaluate the system on public benchmarks with pre-set criteria.

Project structure

The code is split into three Python packages named after the team's original division of work, plus the web app. Everything lives in the repository `ViveekB410/SatyaScore`, branch `v2-crosscheck` (Version 2 work) and `main`.

Folder	What it holds	Key files
`member1_model/`	Models and scoring	`embeddings.py` (XLM-R sentence vectors), `nli_scorer.py` (mDeBERTa inference), `cmhsa.py` (score, thresholds, whole-text verdict), `llm_verifier.py` (LLM calls, cache, Groq-to-local fallback), `calibration.py`, `train_weights.py`
`member2_data/`	Structured sources	`entity_linker.py` + `wikidata_connector.py` + `kg_extra.py` (Wikidata checks, English and Hindi), `official_stats.py` (World Bank, WHO), `factcheck.py` (Google Fact Check API), `kg_relations.py` (extra relations, switched off), `dates.py`, `_http.py` (polite web requests)
`member3_backend/`	The pipeline and the server	`pipeline.py` (runs every step for a request), `claim_extractor.py`, `faiss_retriever.py`, `evidence_rules.py`, `factcheck_rules.py`, `crosscheck.py` (Version 2 rule), `wiki_expander.py` (fetches missing Wikipedia articles), `api.py` (REST API and settings), `auth.py` (accounts), `serve.py` (website + API on one port)

Folder	What it holds	Key files
`web/`	The web app (React + TypeScript)	`src/pages/Analyse.tsx` (input and results), `src/components/FlaggedClaims.tsx` (claim cards), `MarkedText.tsx` (underlined text), `src/lib/api.ts` (calls to the server)
`tests/`	308 automated tests	one file per module, e.g. `test_crosscheck.py`, `test_llm_verifier.py`, `test_pipeline.py`
`v2data/`	Version 2 evaluation	test sets, cached runs and LLM answers, `score.py`, `make_results.py`
`results/`	Every measured result	`v2_results_final.md`, `v2_data_PROTOCOL.md` (all rounds and rule changes)
`deploy/`	Sharing and hosting	`start_public.py` (public link), `azure/` (cloud scripts, not set up)
`deliverables/` (not in git)	Paper, synopsis, report	`SatyaScore_IEEE_Paper_v2`, `SatyaScore_Synopsis_v2`, `SatyaScore_Project_Report_v2`, generator scripts in `docs_v2_src/`

Data files in the project root: `corpus_passages_expanded.json` + `corpus_vectors_expanded.npy` (the 102,069-passage evidence index), `evidence_corpus_cache.json`, `weights_en.json` / `weights_hi.json` (trained CMHSA weights). The account database is `instance/satyascore.db`; web caches are in `instance/wiki_cache` and `instance/factcheck_cache`. The start files are `start_v2.ps1` and `Start Satya-Score.bat` in the project root.

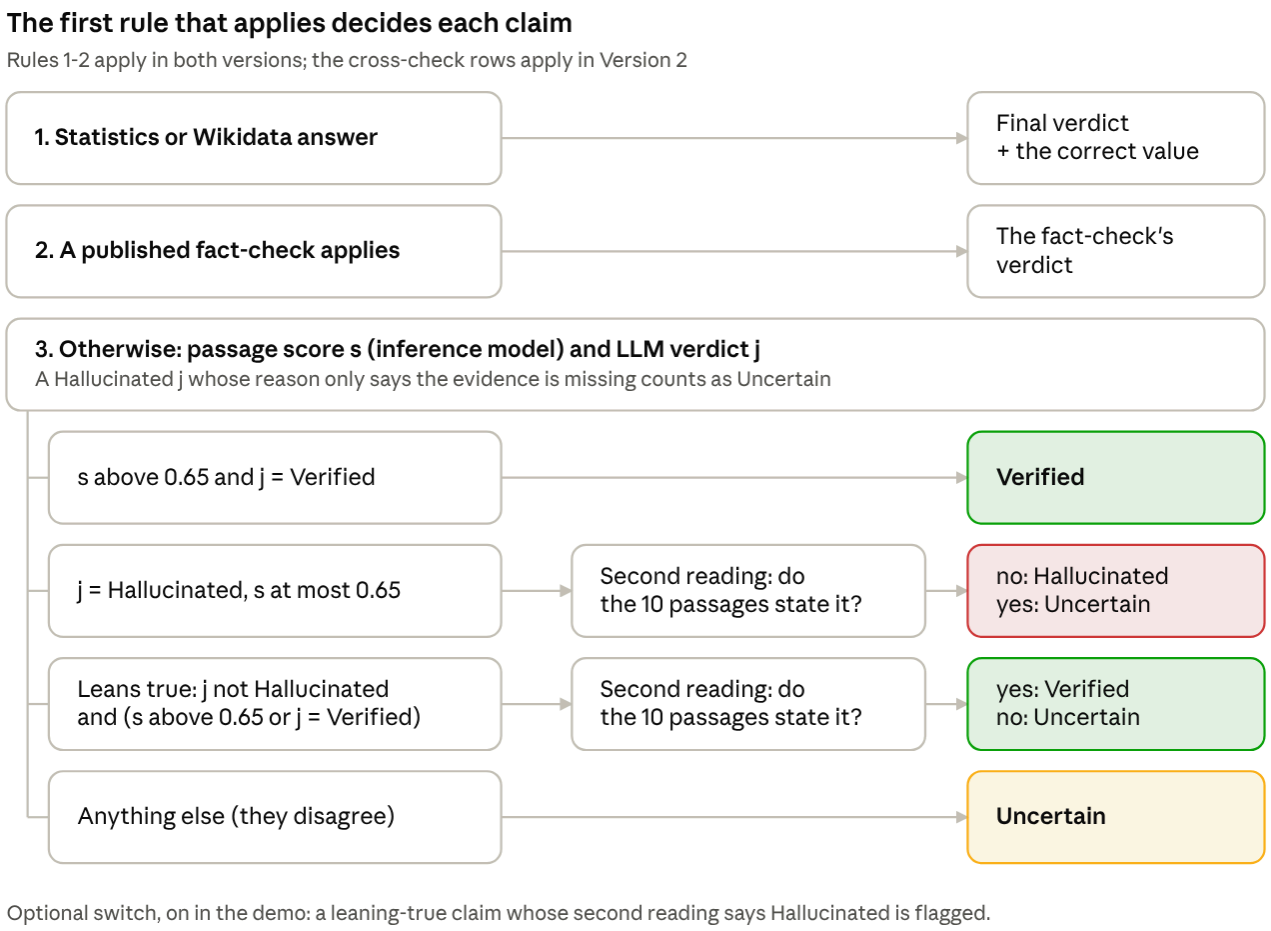
How one request is answered

A check of a 5-claim English answer took 18.2 s in Version 2 with Groq answering (measured today): extraction and look-ups about 5 s, the LLM about 8 s, the rest under 2 s. These are the steps, in the order the code runs them (`member3_backend/pipeline.py`, `run()`):

1. **Browser.** You paste text on the Analyse page and press Analyse. `web/src/lib/api.ts` sends `POST /detect` with the text and the language (Devanagari script means Hindi). The page waits up to 5 minutes.
2. **API gate** (`api.py`). The request is checked: signed-in user (session cookie), developer key, or anonymous with a lower rate limit. Text over 20,000 characters is refused.
3. **Claim extraction** (`claim_extractor.py`). English: spaCy parses each sentence; joined statements are split, list items become claims, and "it"/"he" is replaced by the topic of the passage. Hindi: the Indic NLP Library splits sentences. Questions, greetings, advice and opinions are dropped.
4. **Missing articles** (`wiki_expander.py`). If a claim names a Wikipedia article not yet in the index, it is fetched, embedded and added (cached in `instance/wiki_cache`).
5. **Embedding.** Each claim becomes a vector with the XLM-R sentence encoder (GPU when available).
6. **Structured look-ups, in parallel** (up to 6 at once). For each claim: World Bank and WHO statistics, then Wikidata relations (birthplace, capital, author, founding date and so on). A supported or contradicted answer here is final.
7. **Retrieval** (`faiss_retriever.py`). FAISS returns the 10 nearest passages; those with cosine similarity of at least 0.3 are kept.
8. **Inference, one batched call** (`nli_scorer.py`). mDeBERTa scores every claim-passage pair for entailment and contradiction; the evidence rule turns them into the passage score s (Section 4).
9. **Fact-checks, in parallel** (`factcheck.py` , `factcheck_rules.py`). Google Fact Check results are matched in the right direction; a match decides the claim.
10. **LLM cross-check, Version 2 only** (`crosscheck.py` , `llm_verifier.py`). Claims not yet decided go to gpt-oss-20b: first reading, then, when needed, the second reading of the 10 passages. Groq is tried first, the laptop model takes over at Groq's limit, and answers are cached.
11. **Explanations.** The deciding passage sentence, the Wikidata fact with the correct value, and the wrong-word positions (error spans) are attached.
12. **Whole-text verdict** (`cmhsa.py`). Hallucinated if at least a third of the claims are; otherwise Uncertain if any claim is not Verified; otherwise Verified.
13. **Response.** The API maps error spans back onto your original text, adds the calibrated confidence and the source links, saves the check to History if you are signed in, and returns JSON.
14. **Display** (`Analyse.tsx`). The text is shown with each claim underlined in its verdict colour and the wrong words in bold; below it, one card per claim, grouped Hallucinated, Uncertain, Verified.

The decision rules

Each claim gets the verdict of the first rule that applies; the LLM can never decide a claim alone.



Optional switch, on in the demo: a leaning-true claim whose second reading says Hallucinated is flagged.

decision rules · member3_backend/crosscheck.py and pipeline.py

Passage score s (evidence rule). For the up to 10 retrieved passages, the inference model gives an entailment probability e and a contradiction probability c per passage. If the best e is at least 0.5, s = 0.5 + 0.5 × max e. Otherwise s = 0.5 + 0.5 × (max e − max c over on-topic passages), where a passage is on-topic when its article title appears in the claim. Support can come from any passage; a contradiction only from text about the claim's own subject.

Score and verdict (CMHSA). The designed formula is HalluScore = α·SemSim + β·NLI + γ·KG with α fixed at 0 and trained β ≈ γ ≈ 0.5. In practice the first source decides the score: Wikidata or statistics give 1.0 or 0.0, a fact-check 0.95 or 0.05, otherwise s (Version 1) or, in Version 2, the mean of s and the LLM's probability kept inside the verdict's band. Below 0.35 is Hallucinated, above 0.65 Verified, between them Uncertain.

Safeguards on each source. Wikidata: when several entities share a name, one match is enough to support, but a contradiction needs every namesake with a value to disagree. Statistics: World Bank and WHO must agree when both have a figure, and a claim is judged at the precision it is written in. Fact-checks: a "false" rating transfers only if our claim entails the checked claim, a "true" rating only in the other direction.

Whole text. Hallucinated when at least one third of its claims are Hallucinated; otherwise Uncertain if any claim is Hallucinated or Uncertain; otherwise Verified. The overall score is the mean of the claim scores.

Models, data sources and services

All models are open and run on the laptop; only the look-ups and the optional Groq service need the internet.

Part	What is used	Runs where	Licence / limits
Sentence encoder	paraphrase-xlm-r-multilingual-v1 (XLM-R)	laptop GPU or CPU	Apache 2.0
Inference model	mDeBERTa-v3-base-mnli-xnli	laptop GPU or CPU	MIT
LLM (Version 2)	gpt-oss-20b: 21B parameters, 3.6B active per word	Ollama on the laptop (13 GB, mostly CPU) or Groq cloud	Apache 2.0; Groq free tier 8,000 tokens per minute, 1,000 requests per day
English parsing	spaCy en_core_web_sm	laptop	MIT
Hindi sentence split	Indic NLP Library	laptop	MIT
Vector search	FAISS over 102,069 Wikipedia passages (89,594 English, 12,475 Hindi)	laptop	MIT
Knowledge graph	Wikidata (entity search + SPARQL)	internet	CC0
Statistics	World Bank Open Data; WHO Global Health Observatory	internet	CC BY 4.0; WHO CC BY-NC-SA 3.0 IGO (non-commercial only)

Part	What is used	Runs where	Licence / limits
Fact-checks	Google Fact Check Tools API	internet	needs `GOOGLE_FACTCHECK_KEY`
Encyclopaedia text	English and Hindi Wikipedia (index + articles fetched on demand)	internet for new articles	CC BY-SA 4.0

The API keys (Google Fact Check, Groq) are read from your own environment when the server starts; they are never printed, logged or stored in the project. Without the Google key, fact-checks are skipped; without the Groq key, the laptop model does all LLM work.

Web application and API

The web app (http://localhost:5173) talks to the API (http://localhost:8000) through 12 endpoints; anyone can check text without an account, and signing in adds a saved history.

Pages: Analyse (input and results, `/analyse`), History, Login, Register, Reset password, Account. The start page (`/`) waits until the API answers, then moves on.

What a claim card shows:

- the verdict chip and the claim as checked (with "it" replaced by its subject);
- for a structured source: the correct value, e.g. "Correct fact: भारत · राजधानी: नई दिल्ली (Wikidata)";
- in Version 2, one chip per reader: Passages (support / contradict / unclear, with s), LLM (with its reason), and Second reading or Check before flagging;
- the evidence line: the deciding source as a link, and how many articles were checked.

In the text above the cards, each claim is underlined in its verdict colour and the wrong words of a flagged claim are bold.

Endpoint	Method	Purpose
`/detect`	POST	Check a text: returns claims, verdicts, scores, sources, error spans, LLM readings
`/extract-text`	POST	Read the text out of an uploaded .txt or .pdf
`/health`	GET	Server status, number of passages, mode (`v1` or `v2`)

Endpoint	Method	Purpose
`/auth/register`, `/auth/login`, `/auth/logout`	POST	Accounts and sessions
`/auth/change-password`, `/auth/reset-password`	POST	Password change (needs the old one) and reset with a one-time code
`/auth/me`	GET	Who is signed in
`/history`, `/history/{id}`	GET, DELETE	Saved checks of the signed-in user

Security: passwords are stored only as salted scrypt hashes; session tokens and reset codes only as SHA-256 hashes; sessions use HttpOnly cookies; requests from other websites are refused by an origin check; 5 failed sign-ins in 15 minutes block that account from that address; checks are limited to 100 per minute per user or key and 20 per minute per address for anonymous use. API documentation is at http://localhost:8000/docs.

Running the system

Double-click `Start Satya-Score.bat` in the project folder: it starts both servers in Version 2 and opens http://localhost:5173/analyse in your browser after about a minute.

What the start script (`start_v2.ps1`) does:

1. Opens an **API window** with the Version 2 settings and starts the server on port 8000 (skipped if one already runs).
2. Opens a **website window** with `npm.cmd run dev` on port 5173 (skipped if one already runs).
3. Waits until http://localhost:8000/health answers, then opens the browser. Keep both windows open; closing them stops the app.

To confirm the mode, open http://localhost:8000/health and look for `"mode":"v2"`. A server started by hand with `uvicorn` and no settings runs **Version 1**, which marks more false claims Verified.

Settings (environment variables read when the API starts):

Setting	Version 2 value	Default	Effect
`SATYASCORE_VERDICT_MODE`	`crosscheck`	`v1`	Turns the LLM cross-check on
`SATYASCORE_LLM_PROVIDER`	`groq,ollama` (or `ollama`)	`groq`	LLM services, tried in order

Setting	Version 2 value	Default	Effect
`SATYASCORE_LLM_MODEL`	`openai/gpt-oss-20b,gpt-oss:20b`	per provider	Model name for each provider
`SATYASCORE_FLAG_NEEDS_LLM`	`true`	`false`	A passage contradiction alone never flags; check before flagging
`SATYASCORE_SECOND_READING_FLAGS`	`true` (demo)	`false`	Second reading's "Hallucinated" may flag a claim (not the tested rule)
`SATYASCORE_SECOND_READING`	(default)	`true`	Second reading on or off
`SATYASCORE_LLM_WORKERS`	(default)	1 for local only, 4 with a cloud provider	Claims read at once
`SATYASCORE_OFFICIAL_STATS`	(default)	`true`	World Bank / WHO checks
`SATYASCORE_KG_RELATIONS`, `_NUMBER_GUARD`, `_MEDLINE`	(default)	off	Tested and not adopted
`SATYASCORE_API_KEYS`, `SATYASCORE_DB`	(default)	dev key; `instance/satyascore.db`	API keys, account database

Speed (RTX 4050 laptop, measured):

Configuration	First check	Same text again
Version 1, 6 claims	5.1 s	0.7 s
Version 2, laptop LLM only, 6 claims	45.7 s	under 3 s
Version 2, Groq first, 5 claims	18.2 s	2.7 s

The first check after starting is slower by about half a minute while the 13 GB model loads. Groq's free tier allows about 3–4 readings a minute; the rest fall to the laptop model.

Troubleshooting:

- *Page says Offline or Retry:* the API window is closed or still loading; wait for "Application startup complete".

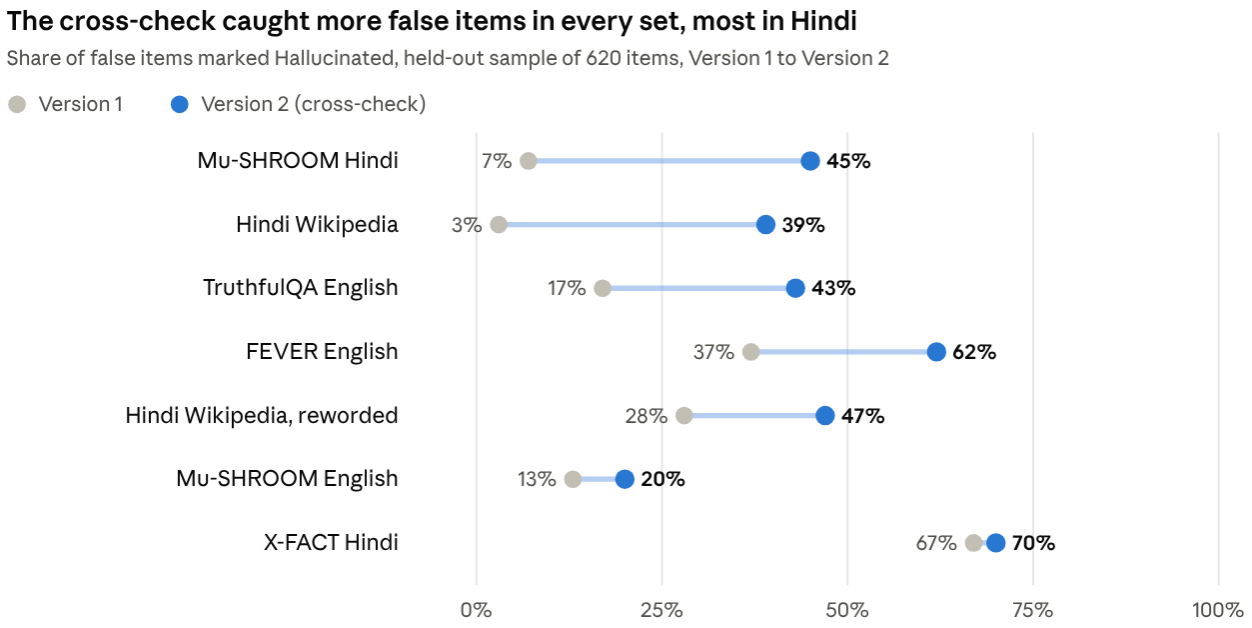
- *Page loads but nothing connects:* the website window is missing; run the batch file again (it starts only what is missing).
- *False claims marked Verified:* check `/health`; `"mode":"v1"` means the server was started without the Version 2 settings.
- *Very slow or servers vanish:* memory is short (the LLM needs about 13 GB); close Chrome tabs and other large apps.
- *The public link:* `python deploy\start_public.py` gives a temporary https link while the laptop stays on; it serves the built website, so rebuild after changes (`npm run build` in `web`).

Evaluation and results

On 620 held-out items run once, Version 2 caught 47% of false items against 25% and passed 6% as Verified against 22%, with false alarms at 13% against 11%. It met 3 of the 4 criteria fixed before the run.

Measure	Version 1	Version 2	Criterion fixed in advance	Met
False items caught	95/374 (25%)	176/374 (47%)	at least 2× Version 1	no (1.85×)
False items passed as Verified	83/374 (22%)	23/374 (6%)	at most half	yes
True items Verified	137/246 (56%)	139/246 (57%)	at most 5 points lower	yes
False alarms on true items	28/246 (11%)	32/246 (13%)	at most 3 points higher	yes

The gains in catches and in false items no longer passed are significant (McNemar $p < 0.001$); the false-alarm change is not ($p = 0.63$). The LLM alone caught 86% but raised false alarms on 28% of true items.



results/v2_results_final.md · caught per set, 620 held-out items

The cost sits in TruthfulQA (false alarms 18% → 27%, answers lack their question) and the reworded Hindi sentences (3% → 11%). X-FACT was already caught by published fact-checks.

How it was tested. No label was written by the team: FEVER, Mu-SHROOM (a claim is false if it overlaps a human-marked span by 3+ characters), TruthfulQA, X-FACT, and Hindi Wikipedia sentences with one number changed (as written and reworded by Qwen). The LLM was chosen on development data (gpt-oss-20b: balanced accuracy 57%, false alarms 7%). Four held-out rounds were run, each on a new sample and each once: 201, 175, 162 and finally 620 items. Rounds 1 and 3 failed; the rule changes between rounds are recorded in `results/v2_data_PROTOCOL.md` before each run.

Version 1 component results (still part of the system): altered country statistics caught 137 of 137, no true figure contradicted; published fact-checks made 0 wrong decisions on 40 claims; Wikidata decisions 11 of 13 correct (pre-registered), 12 of 12 after a linking fix (exploratory); adding the knowledge graph raised AUC from 0.704 to 0.784 in a pre-registered ablation; on chatbot claims Version 1 flagged 9 of 258 true claims but caught only 5 of 45 hallucinations, which is why Version 2 was built.

System check (13 October 2026, evening)

Everything passed: 319 automated tests, the website build, all data files, and two live checks in which all 7 claims got the right verdict.

Check	Result
Automated tests (`pytest`)	319 passed
Website type-check and production build	passed, built in 2 s
Evidence index	`corpus_passages_expanded.json` 84.8 MB + vectors 299 MB present; 102,216 passages loaded (grows as articles are fetched)
Weights, database, start files	`weights_en.json` , `weights_hi.json` , `instance/satyascore.db` , `start_v2.ps1` , `Start Satya-Score.bat` present
Local LLM	`gpt-oss:20b` (13 GB) installed in Ollama, Ollama running
API health	`{"status":"ok","mode":"v2"}` on port 8000
Website server	not running at the time of the check: start it with the batch file
Deliverables	IEEE paper v2, synopsis v2, project report v2 (PDF and Word) in `deliverables/`
Git	Version 2 is on `main` (commits `784bb99` and `655b2ef` , pushed); one uncommitted change: the improved `start_v2.ps1`

Live check, English (13.2 s, Groq answering):

Claim	Verdict	Decided by
India's population is about 1.4 billion.	Verified	cross-check: passages and LLM agree
The capital of Australia is Sydney.	Hallucinated	Wikidata (correct: Canberra)
Photosynthesis takes place in the chloroplasts of plant cells.	Verified	cross-check
Napoleon Bonaparte was born in Paris.	Hallucinated	Wikidata (correct: Ajaccio)

Live check, Hindi (9.3 s):

Claim	Verdict	Decided by
गंगा नदी हिमालय से निकलती है।	Verified	cross-check
महात्मा गांधी का जन्म पोरबंदर में हुआ था।	Verified	Wikidata
सूरज पश्चिम से उगता है।	Hallucinated	cross-check, confirmed by the second reading

Earlier today's checks found and fixed four problems: the web page gave up after 90 s (now 5 minutes); a server started without settings silently ran Version 1 (now `/health` reports the mode and the start script sets it); the website window failed to start because Windows blocked `npm` (now `npm.cmd`); and a claim card said "none clearly contradicts" next to a "contradict" chip (now consistent).

Limitations and honest caveats

State these yourself before a reviewer finds them; every one is already written in the paper and the report.

- **About half of the false items are still missed.** Version 2 catches 47%; when the passages and the LLM disagree, the claim is left Uncertain on purpose.
- **The doubling target was missed** (1.85×, needed 2×), so Version 1 remains the code default. Never call Version 2 "adopted"; say it met 3 of 4 pre-set criteria.
- **The criterion was changed once** after the first round failed (from an absolute 5% false-alarm limit to one relative to Version 1), and the final rule was chosen on samples already used. Both are recorded in the protocol. All held-out samples are now used up, so no further tuning can be tested honestly.
- **Test data caveats:** constructed Hindi false items change only one number; Mu-SHROOM claim labels come from span overlap; TruthfulQA answers lack their question; at most 60 true and 60 false items per set.
- **The second-reading flag switch** (on in the demo) was not part of the tested rule: on the cached test answers it raised catches 47% → 49% and false alarms 13% → 15%.
- **Speed:** 16–46 s per answer on the laptop; Groq's free tier runs out after 3–4 readings a minute.
- **Memory:** the local LLM needs about 13 GB of RAM; with Chrome open the system can kill the servers.
- **Hindi pronouns** are not resolved: "इसे शाहजहाँ ने बनवाया था" stays Uncertain because "इसे" is not linked to the Taj Mahal.

- **Not permanently online:** the public link works only while the laptop runs; the Azure scripts exist but were never set up.
- **Labelled chatbot data are small** (456 claims, 9 Hindi hallucinations), each labelled by one of the four authors with an AI suggestion visible (blind agreement $\kappa = 0.53$).
- **WHO data** may be used for non-commercial purposes only.
- **Pending from you:** author e-mails in the IEEE paper, HOD and Principal names on the report's certificate page, resumes, and the HOD's feedback on the synopsis.

Viva and review questions

Short answers you can give in your own words; each is backed by a number in this document.

Question	Short answer
What is new in your project?	The combination: claim-level checking in English and Hindi, structured sources first with the correct value shown, and an LLM that can only count when the evidence agrees. No single model or source is ours.
Why not just ask an LLM?	Alone it caught 86% of false items but flagged 28% of true ones. Requiring agreement with the passages brought false alarms to 13%.
Why do you need the inference model if you have an LLM?	It gives an independent, reproducible reading of the actual passages. The LLM can rely on memory; the two must agree for Verified.
Why is a false claim sometimes Uncertain, not Hallucinated?	The readers disagreed (e.g. passages seemed to support, LLM said false). We prefer Uncertain to a wrong verdict either way.
What does CMHSA do now?	It sets the score scale and thresholds (0.35 / 0.65). Similarity was dropped ($\alpha = 0$) because it measures topic, not truth; a structured answer overrides the weighted sum.
How did you avoid tuning on test data?	Choices on development data; each held-out sample used once; criteria and changes written in `results/v2_data_PROTOCOL.md` before each run.

Question	Short answer
Did Version 2 pass?	3 of 4 criteria: catches rose 1.85× (needed 2×); false items passed as Verified fell from 22% to 6%; false alarms 11% → 13%, not significant.
Why gpt-oss-20b?	Best balanced accuracy in a bake-off fixed in advance (57%, false alarms 7%); open weights, Apache 2.0, runs on the laptop.
How is Hindi handled?	Indic NLP sentence split, a multilingual encoder and inference model, Hindi Wikipedia passages, Hindi Wikidata templates; the LLM reads Hindi directly.
Is user data safe?	Locally, nothing leaves the laptop except look-ups; with Groq, claims and passages go to Groq. Passwords are scrypt-hashed; keys are never stored.
Why is it slow?	The 13 GB model does not fit the 6 GB GPU, so most of it runs on the CPU (about 4 s per reading). Groq or a hosted model fixes this.
What would you do next?	More catches without more false alarms, faster hosting, Marathi, and a larger Hindi set labelled blind by two people.

Glossary

Term	Meaning in this project
Claim	One self-contained factual statement taken from the answer
Hallucination	A false or unsupported statement written confidently by an LLM
Verified / Uncertain / Hallucinated	Supported by evidence / not settled / contradicted; score above 0.65 / 0.35–0.65 / below 0.35
Caught	A false item marked Hallucinated
False alarm	A true item marked Hallucinated
Passed as Verified	A false item wrongly marked Verified

Term	Meaning in this project
NLI (inference model)	A model that says whether a passage entails or contradicts a claim (mDeBERTa)
Passage score s	The evidence rule's combination of the passages' entailment and contradiction
On-topic passage	A passage whose article title appears in the claim; only these may contradict
CMHSA	Composite Multilingual Hallucination Scoring Algorithm: the score formula, thresholds and whole-text rule
Cross-check	Version 2 rule: passages and LLM must agree for Verified; the LLM must say false to flag
First reading / second reading	LLM with top-5 passages and its own knowledge / LLM with all 10 passages only
No-evidence filter	An LLM "false" whose reason only says the evidence is missing counts as Uncertain
Error span	The wrong words of a flagged claim, shown in bold in the text
FAISS	The library that finds the passages nearest to a claim
Wikidata	Open knowledge graph of facts (birthplaces, capitals, dates)
Ollama / Groq	Runs the LLM on the laptop / runs the same model in the cloud
Held-out sample	Test items never used while building, each run once
McNemar test	A test of whether two systems differ on the same items
Pre-registered	Criterion written down before the experiment was run